



JS-1

JavaScript Introduction + ES6+

let · const · arrow functions · template literals · destructuring · rest / spread

MERN Stack Training Program

Trainer's Master Teaching Guide — First Principles to Advanced

Theory • Real-World Examples • Code Walkthroughs • Mini Project • Practice Set

Prepared for: MyAnatomy Trainers

Table of Contents

- | | |
|----|--|
| 1 | Introduction to JavaScript — What, Why, How |
| 2 | Variables Deep Dive — var, let, const |
| 3 | Arrow Functions — Syntax to 'this' Binding |
| 4 | Template Literals — Modern String Handling |
| 5 | Destructuring — Objects and Arrays |
| 6 | Rest & Spread Operators |
| 7 | Mini Project — Student Report Card Generator |
| 8 | Practice Assignment for Students |
| 9 | One-Page Cheat Sheet |
| 10 | Likely Interview / Viva Questions |
-

1. Introduction to JavaScript — What, Why, How

1.1 What is JavaScript?

JavaScript (JS) is a high-level, interpreted (technically **JIT-compiled**), dynamically-typed programming language originally built to make web pages interactive. It was created by **Brendan Eich at Netscape in 1995** — famously written in about **10 days**. Despite the rushed origin, it has grown into one of the three foundational technologies of the web, alongside HTML and CSS.

Why does this history matter to a student? Because it explains JS's quirks (like type coercion, the `var` keyword's odd scoping) — they are not "bad design", they are the result of a language that had to grow up in public while millions of websites already depended on it. ES6 (2015) was the biggest turning point — it modernized JS without breaking the old web.

1.2 Where does JavaScript run?

- **Browser (Client-side):** Every browser has a JS Engine — Chrome/Node use **V8**, Firefox uses **SpiderMonkey**. This is how JS manipulates web pages (DOM), handles clicks, validates forms.
- **Server-side (Node.js):** In 2009, Ryan Dahl took Chrome's V8 engine and let it run **outside the browser** — this is Node.js. This single innovation is *why the 'N' in MERN exists*.

WHY THIS MATTERS FOR MERN: This is the single most important sentence for a MERN student: **MERN = MongoDB, Express, React, Node — and all four pieces of that stack are held together by one language: JavaScript.** Learn JS deeply once, and you can write the database queries, the server API, and the user interface, without context-switching languages.

1.3 How JS is added to a page (quick recap before we go further)

```
<!-- 1. Inline JS (avoid in real projects) -->
<button onclick="alert('Hi')">Click</button>

<!-- 2. Internal JS -->
<script>
  console.log("Running inside the HTML file");
</script>

<!-- 3. External JS (industry standard, used in MERN/React) -->
<script src="app.js" defer></script>
```

Fig 1.1 — Three ways to attach JavaScript to a page

1.4 Core nature of JavaScript (say this out loud in class)

Property	Meaning	Real-world consequence
Interpreted / JIT	Code runs without a separate compile step	You can test instantly in console — huge for fast prototyping
Dynamically typed	A variable's type is decided at runtime	Flexible but bug-prone — motivates tools like TypeScript later

Property	Meaning	Real-world consequence
Single-threaded	One command executes at a time	Explains why async (Promises, async/await) exists — covered in a later session
Prototype-based	Objects inherit from other objects, not classes	Explains why JS "classes" (ES6) are just syntax sugar

Live Demo for Class

Open Chrome DevTools (F12) → Console tab. Type `typeof "hello"`, then `typeof 5`, then reassign a variable to a different type. This 30-second demo makes "dynamically typed" click instantly — far better than a definition on a slide.

2. Variables Deep Dive — var, let, const

2.1 Why this topic exists at all

Before ES6 (2015), JavaScript had only one way to declare a variable: `var`. It caused real, repeated bugs in production code at companies worldwide. ES6 introduced `let` and `const` specifically to fix those bugs. Teach the problem first — the solution will then feel obvious rather than arbitrary.

2.2 The problem with var: Function Scope, not Block Scope

```
if (true) {  
  var age = 25;  
}  
console.log(age); // 25 -> "leaked" outside the if-block!  
  
{  
  let city = "Delhi";  
}  
console.log(city); // ReferenceError: city is not defined (correctly blocked)
```

Fig 2.1 — `var` escapes block scope; `let` does not

Why: `var` is scoped to the nearest *function* (or globally, if not in any function). It ignores `{ }` blocks belonging to `if`, `for`, etc. `let` and `const` are scoped to the nearest *block* — matching what every student intuitively expects from curly braces.

2.3 The classic real-world bug: var inside a loop

This is the single most famous JavaScript interview question and it explains the entire `var` vs `let` debate in one demo.

```
// Real-world scenario: printing button numbers after 1 second, using var  
for (var i = 1; i <= 3; i++) {  
  setTimeout(() => console.log("Button", i), 1000);  
}  
// Output after 1 second: Button 4, Button 4, Button 4 <-- BUG!  
  
// Same code using let  
for (let i = 1; i <= 3; i++) {  
  setTimeout(() => console.log("Button", i), 1000);  
}  
// Output after 1 second: Button 1, Button 2, Button 3 <-- CORRECT
```

Fig 2.2 — The interview-favourite `var/let` loop bug

Reasoning: With `var`, there is only **one shared *i*** across the whole loop (function-scoped) — by the time the callbacks run, the loop has already finished and *i* is 4. With `let`, JS creates a **fresh *i* for every single iteration** (block-scoped) — so each callback remembers its own value. This directly maps to real MERN bugs: rendering a list of buttons in React where every click handler must "remember" its own item.

2.4 const — not "constant value", but "constant binding"

A very common misconception: students think `const` means the value can never change. It actually means the **variable cannot be reassigned** to point to something else. If the value is an object or array, its *contents* can still be mutated.

```
const student = { name: "Riya", marks: 80 };
student.marks = 95;      // OK - mutating a property, not reassigning
console.log(student);    // { name: "Riya", marks: 95 }

student = { name: "New" }; // TypeError: Assignment to constant variable.

const scores = [10, 20];
scores.push(30);         // OK - array contents can change
scores = [1, 2, 3];      // TypeError
```

Fig 2.3 — `const` locks the binding, not the object's contents

2.5 Hoisting and the Temporal Dead Zone (TDZ)

"Hoisting" means JS moves variable and function *declarations* to the top of their scope during a pre-processing phase, before the code actually runs.

```
console.log(x); // undefined (not an error!)
var x = 10;

console.log(y); // ReferenceError: Cannot access 'y' before initialization
let y = 20;
```

Fig 2.4 — `var` hoists as undefined; `let/const` hoist into the Temporal Dead Zone

Reasoning: `var` is hoisted *and initialised* with undefined immediately. `let/const` are hoisted but **not initialised** — the zone between the top of the scope and the actual declaration line is called the **Temporal Dead Zone**. Accessing the variable there throws an error. This is a deliberate safety feature, not a limitation.

2.6 Comparison Table (put this on the whiteboard)

Feature	var	let	const
Scope	Function	Block	Block
Redeclare in same scope	Allowed	Error	Error
Reassign value	Allowed	Allowed	Not allowed
Hoisting behaviour	Hoisted, = undefined	Hoisted, in TDZ	Hoisted, in TDZ
Attached to window (browser global)	Yes	No	No
Recommended usage	Avoid in modern code	When value will change	Default choice

2.7 Best-Practice Rule to give students

RULE OF THUMB: Default to **const**. Switch to **let** only when you know the variable's value must change (counters, loop indices, reassigned totals). Treat **var** as legacy — you'll see it in old code and interview questions, but don't write new code with it.

3. Arrow Functions — Syntax to 'this' Binding

3.1 Why arrow functions were introduced

Two problems motivated arrow functions in ES6: (1) traditional function syntax is verbose for small callbacks, which are extremely common in JS (array methods, event handlers, promises); (2) the behaviour of the `this` keyword inside a regular function was a constant source of bugs. Arrow functions solve both.

3.2 Syntax evolution — teach it as a gradual transformation

```
// Traditional function expression
function add(a, b) {
  return a + b;
}

// Step 1: convert to arrow function
const add1 = (a, b) => {
  return a + b;
};

// Step 2: single-expression body -> implicit return (no braces, no return)
const add2 = (a, b) => a + b;

// Step 3: exactly one parameter -> parentheses become optional
const square = n => n * n;

// Step 4: zero parameters -> empty parentheses are required
const greet = () => "Hello!";

// Returning an OBJECT with implicit return needs extra ( )
const makeUser = (name, age) => ({ name: name, age: age });
```

Fig 3.1 — From function keyword to fully-shortened arrow syntax

3.3 The real reason arrow functions matter: lexical 'this'

In a regular function, `this` is decided by *how the function is called* (dynamic binding) — which is confusing and inconsistent. An arrow function has **no `this` of its own**; it simply uses the `this` value of the surrounding (enclosing) code, called **lexical `this`**.

```
const trainer = {
  name: "Aman",
  students: ["Riya", "Kabir", "Sara"],

  // Regular function used as a method: 'this' works correctly here
  greetStudents: function () {
    console.log("Trainer:", this.name); // "Aman" - called as trainer.greetStudents()

    // BUG: regular function inside forEach loses 'this'
    this.students.forEach(function (s) {
      console.log(this.name, "teaches", s); // this.name is undefined!
    });

    // FIX: arrow function inherits 'this' from greetStudents
    this.students.forEach((s) => {
      console.log(this.name, "teaches", s); // "Aman teaches Riya" etc. - correct!
    });
  }
};

trainer.greetStudents();
```

Fig 3.2 — Why arrow functions fix the classic 'this' bug inside callbacks

Reasoning: When `forEach` calls the regular callback, it calls it as a plain function (not as `trainer.something()`), so inside that callback `this` is undefined (in strict mode / modules). The arrow function has no `this` of its own, so it looks one level up — to `greetStudents`'s `this`, which correctly refers to `trainer`.

3.4 When NOT to use arrow functions

- **As object methods** that need `this` to refer to the object — use a regular function/method shorthand instead (as shown with `greetStudents` above).
- **As constructors** — arrow functions cannot be used with the `new` keyword; they will throw an error.
- **When you need the special arguments `object`** — arrow functions don't have their own arguments; use rest parameters instead (Section 6).

WHY THIS MATTERS FOR MERN: In React (which every MERN student meets next), this is exactly why **event handlers written as class methods lose** `this` unless bound — and why modern React function components + arrow functions largely sidestep the whole problem. Say this line in class; it creates a strong "aha" connecting today's lesson to next week's React module.

4. Template Literals — Modern String Handling

4.1 The old, painful way

```
var name = "Riya";  
var marks = 95;  
console.log("Hello " + name + ", you scored " + marks + " marks. " +  
  "Grade: " + (marks >= 90 ? "A+" : "A"));
```

Fig 4.1 — String concatenation before ES6: easy to break, hard to read

Every + is a place a mistake can happen (missing space, wrong quote, wrong type). Multi-line strings needed ugly \n or string-joining tricks.

4.2 Template literal syntax — backticks + \${}

```
const name = "Riya";  
const marks = 95;  
  
const message = `Hello ${name}, you scored ${marks} marks. Grade: ${marks >= 90 ? "A+"  
: "A"}`;   
console.log(message);  
  
// Multi-line strings - no special characters needed  
const receipt = `  
  Student: ${name}  
  Marks:   ${marks}  
  Status:  Passed  
`;
```

Fig 4.2 — Backticks (`), \${} interpolation, and free multi-line support

Reasoning: Anything inside \${ } is treated as a live JavaScript expression — a variable, a calculation, even a function call or ternary. This removes the need for + concatenation entirely, and backticks preserve line breaks exactly as typed.

4.3 Real-world MERN examples

```
// 1. Building a dynamic API URL (very common in the M and E of MERN)
const userId = 42;
const url = `https://api.myanatomy.com/users/${userId}/profile`;

// 2. Building HTML dynamically before React (or inside React with JSX-like logic)
const card = `
  <div class="card">
    <h3>${name}</h3>
    <p>Marks: ${marks}</p>
  </div>
`;

// 3. Building a MongoDB-style dynamic query string / log message
console.log(`[${new Date().toISOString()}] User ${name} logged in`);
```

Fig 4.3 — Template literals used for URLs, HTML, and logs

4.4 Advanced (mention briefly, don't deep-dive today): Tagged Templates

A function can be placed right before a template literal to process it (used by libraries like styled-components in React). Just tell students this exists — full depth is for a later advanced session.

```
function shout(strings, ...values) {
  return strings.raw.join("").toUpperCase() + values.join(",");
}
shout`hello ${name}`; // used heavily by styled-components in React later
```

Fig 4.4 — Tagged templates preview

5. Destructuring — Objects and Arrays

5.1 Why destructuring exists

Before ES6, pulling values out of objects/arrays meant repeating the object name again and again. Destructuring lets you "unpack" values into standalone variables in a single, readable line — and it is used *everywhere* in modern MERN code (React props, API responses, Express request objects).

5.2 Object Destructuring

```
const student = { name: "Kabir", age: 21, city: "Pune" };

// Old way
const name1 = student.name;
const age1 = student.age;

// Destructuring way - variable names must match property names
const { name, age, city } = student;
console.log(name, age, city); // Kabir 21 Pune

// Renaming while destructuring
const { name: studentName } = student;
console.log(studentName); // Kabir

// Default value if the property does not exist
const { country = "India" } = student;
console.log(country); // India

// Nested object destructuring
const user = { id: 1, address: { pin: 411001, city: "Pune" } };
const { address: { pin } } = user;
console.log(pin); // 411001
```

Fig 5.1 — Object destructuring: basic, rename, default, nested

5.3 Array Destructuring

```
const marks = [95, 88, 76];

// Basic - position matters, not name
const [first, second, third] = marks;
console.log(first, second, third); // 95 88 76

// Skipping elements with empty commas
const [, , thirdOnly] = marks;
console.log(thirdOnly); // 76

// Default values
const [a, b, c, d = 0] = marks;
console.log(d); // 0 (marks has no 4th element)

// Classic real-world use: swapping two variables in ONE line
let x = 5, y = 10;
[x, y] = [y, x];
console.log(x, y); // 10 5
```

Fig 5.2 — Array destructuring: position-based, skipping, defaults, swap trick

5.4 Destructuring in function parameters (heavily used in React & Express)

```
// Instead of this:
function printUser(user) {
  console.log(user.name, user.age);
}

// Destructure directly in the parameter list:
function printUser({ name, age }) {
  console.log(name, age);
}
printUser({ name: "Sara", age: 22 });

// Real MERN example - Express route handler reading from req.body
app.post("/register", (req, res) => {
  const { username, email, password } = req.body;
  // ...
});

// Real MERN example - React functional component reading props
function StudentCard({ name, marks }) {
  return `${name} scored ${marks}`;
}
```

Fig 5.3 — This exact pattern appears in nearly every Express and React file students will write

WHY THIS MATTERS FOR MERN: Tell students explicitly: "The moment you write `const { name, marks } = req.body` in Express, or `function Card({ title })` in React — that is this exact topic. If destructuring feels shaky today, it will block you in every future class."

6. Rest & Spread Operators

6.1 Same symbol (...), two opposite jobs

Both use three dots The rule to give students: **Spread "expands" a collection into individual items. Rest "collects" individual items into a collection.** Which one is happening depends entirely on *where* the dots appear.

6.2 Spread Operator (...) — expanding

```
// Spreading an array - copy it (without spread, arrays copy by reference!)
const original = [1, 2, 3];
const copy = [...original, 4, 5];
console.log(copy); // [1, 2, 3, 4, 5]

// Spreading into a function call
function total(a, b, c) { return a + b + c; }
const nums = [10, 20, 30];
console.log(total(...nums)); // 30 -> nums is expanded into 3 arguments

// Spreading an object - merge / copy / override properties
const baseUser = { name: "Riya", role: "student" };
const updatedUser = { ...baseUser, role: "mentor" }; // override just one field
console.log(updatedUser); // { name: "Riya", role: "mentor" }
```

Fig 6.1 — Spread: copying arrays, passing as arguments, merging objects

WHY THIS MATTERS FOR MERN: In React, you will very soon write `setState({ ...state, marks: 95 })`. This is spread being used for **immutability** — creating a brand-new object instead of mutating the old one, which is a core rule in React. This single line is one of the most-repeated patterns in the entire MERN stack.

6.3 Rest Parameter (...) — collecting

```
// Collecting unlimited function arguments into a real array
function sum(...numbers) {
  return numbers.reduce((acc, n) => acc + n, 0);
}
console.log(sum(1, 2, 3, 4)); // 10 - works for any number of arguments

// Rest while destructuring - "give me the first one, and the rest separately"
const marksList = [95, 88, 76, 60];
const [topScore, ...remaining] = marksList;
console.log(topScore, remaining); // 95 [88, 76, 60]

const { name, ...otherDetails } = { name: "Sara", age: 22, city: "Pune" };
console.log(name, otherDetails); // Sara { age: 22, city: 'Pune' }
```

Fig 6.2 — Rest: variadic functions and "the rest of it" destructuring

6.4 Spread vs Rest at a glance

	Spread	Rest
Direction	Breaks a collection apart	Gathers items together
Typical location	Inside <code>[]</code> , <code>{ }</code> , or a function call	In function parameters or on the left of destructuring
Example	<code>[...arr]</code>	<code>function f(...args)</code>
Mental model	"Unpack everything here"	"Whatever is left, put it here"

7. Mini Project — Student Report Card Generator

Build this live, from an empty file, using every concept covered today. Narrate each decision out loud: "I'm using const here because this array won't be reassigned... I'm destructuring here because I only need two of these five fields..." — the narration is what teaches reasoning, not just syntax.

Step 1 — Data (const, arrays of objects)

```
const students = [
  { name: "Riya", scores: [92, 88, 95] },
  { name: "Kabir", scores: [70, 65, 60] },
  { name: "Sara", scores: [85, 90, 78] },
];
```

Step 2 — Helper using arrow function + rest + spread

```
// Rest parameter: accept any number of scores
const average = (...scores) => {
  const total = scores.reduce((sum, s) => sum + s, 0);
  return total / scores.length;
};

// Spread: pass an array into a function that expects individual arguments
const getAverage = (scoresArray) => average(...scoresArray);
```

Step 3 — Grading logic + template literal report

```
const getGrade = (avg) => {
  if (avg >= 90) return "A+";
  if (avg >= 75) return "A";
  if (avg >= 60) return "B";
  return "Needs Improvement";
};

// Destructuring inside the function parameter (name, scores pulled out directly)
const buildReport = ({ name, scores }) => {
  const avg = getAverage(scores);
  const grade = getGrade(avg);
  return `Student: ${name} | Average: ${avg.toFixed(1)} | Grade: ${grade}`;
};
```

Step 4 — Put it all together

```
students.forEach((student) => {  
  console.log(buildReport(student));  
});  
  
/* Output:  
Student: Riya | Average: 91.7 | Grade: A+  
Student: Kabir | Average: 65.0 | Grade: B  
Student: Sara | Average: 84.3 | Grade: A  
*/  
  
// Bonus: use spread to add a new student without touching the original array  
const withNewStudent = [...students, { name: "Aman", scores: [100, 100, 99] }];  
console.log(buildReport(withNewStudent[withNewStudent.length - 1]));
```

Fig 7.1 — Complete working mini project combining all six concepts from today

TIE IT BACK: This exact shape — array of objects, transformed with small pure functions, rendered as strings — is precisely what a React component does when it renders a list, and what an Express route does when it formats a JSON response. Tell students this out loud at the end of the demo.

8. Practice Assignment for Students

Give these as homework. They increase in difficulty and force students to combine concepts, exactly like the mini project above.

- 1 Declare a const object car with brand, model, price. Try reassigning the whole object (observe the error) and then changing just price (observe it works). Explain why in one sentence.
- 2 Rewrite this function as an arrow function with implicit return: `function double(n) { return n * 2; }`
- 3 Create a template literal that builds a sentence: "<name> is <age> years old and lives in <city>." using at least 3 variables.
- 4 Given `const point = { x: 10, y: 20, z: 30 }`, destructure only x and z in one line.
- 5 Given `const colors = ["red", "green", "blue", "yellow"]`, use array destructuring with rest to get the first color separately and the remaining three as an array.
- 6 Write a function `sumAll(...nums)` using the rest parameter that returns the sum of any number of arguments passed to it.
- 7 Given two objects `defaults = { theme: "light", fontSize: 14 }` and `userPrefs = { fontSize: 18 }`, use spread to merge them so userPrefs overrides defaults.
- 8 **Challenge:** Extend today's Student Report Card mini project to also print the topper (highest average) using the concepts learned today only.

9. One-Page Cheat Sheet

```
// VARIABLES
const PI = 3.14;           // block-scoped, cannot be reassigned
let count = 0;             // block-scoped, can be reassigned
// var x = 1;              // AVOID - function-scoped, hoists as undefined

// ARROW FUNCTIONS
const add = (a, b) => a + b; // implicit return
const square = n => n * n;   // 1 param, no parens needed
const log = () => console.log("hi"); // 0 params need ( )
// Arrow functions do NOT have their own `this` - they inherit it

// TEMPLATE LITERALS
const msg = `Hello ${userName}, total: ${price * qty}`;

// DESTRUCTURING
const { name, age } = person; // object
const [first, second] = list; // array
function show({ title, price }) { ... } // in parameters

// SPREAD (expand)           // REST (collect)
const copy = [...arr];       function f(...args) {}
const merged = { ...a, ...b }; const [x, ...rest] = arr;
```

Fig 9.1 — Print this page and keep it visible while teaching

10. Likely Interview / Viva Questions

Share these with students at the end of class so they can self-test. Encourage them to answer out loud, not just read.

- 1 What is the difference between `var`, `let`, and `const`?
- 2 What will this print, and why? A `for` loop with `var i` and a `setTimeout` inside it.
- 3 What is the Temporal Dead Zone?
- 4 Can you reassign a property of a `const` object? Can you reassign the object itself?
- 5 How does `this` behave differently inside an arrow function versus a regular function?
- 6 Why can't arrow functions be used as constructors?
- 7 What is the difference between the spread operator and the rest parameter, given they use the same `...` syntax?
- 8 How would you merge two objects while letting the second one's properties win on conflicts?
- 9 How would you swap two variables without a temporary third variable, using ES6?
- 10 Why are template literals generally preferred over string concatenation with `+`?

End of JS-1 Trainer Guide — MyAnatomy. Good luck for tomorrow's session, Sir. Teach the why, and the how will follow.